

● 读完了代码。基于当前架构（manifest-based safe-overwrite + 分层 templates + 多 target），我按价值/风险/战略契合度三个维度规划了四个阶段。

规划：harness 功能扩展路线图

设计约束（先锚定再提建议）

项目的设计哲学已经在 docs/architecture.md 和 README 里明确：

- 薄 provider 适配层 — 不做大一统
- manifest-based 安全覆盖 — 不做 3-way merge
- docs 反映现实 — 不造假功能
- 私有分发优先 — 不追求公网 npm

所有新功能必须符合这些约束，否则就是架构偏移。

Phase 1 — 可观测性与安全回退（最高优先级）

当前 manifest 记录了所有文件的哈希，但用户看不到这个信息。这是最大的 UX 缺口。四个命令填满这个回路，全部是 manifest 的自然延伸：

命令	作用	价值	风险
列出所有受管文件，标注	status unchanged /	高 — 当前只能通过 update	零（只读）
locally-modified / missing	--dry-run 间接窥探		
/ drifted			
diff 显示本地修改文件 vs	高 — 今天 --force 是全有或全无，	零（只读） 模板版本的 unified diff	用户根本不知道会丢什么
— restore 把单个文件恢复到模板版本，	高 — 补齐 diff 之后的自然动作	中（破坏本地修改，	
更新 manifest	需 confirm）		
uninstal 按 manifest 精确删除，保留	中高 — 切换 target	高（破坏性，需	user-added 文件 或干净重置必需；目前只能 rm -rf dry-run 默认 +
			~/claude/ 把本地工作一起炸掉 确认）

实施要点：全部可以在 src/lib/template.js 现有 API 之上加薄壳，不动核心。每个命令独立可发。

预估工作量：每个命令 ~100 行实现 + ~80 行测试，合计 1-2 天。

Phase 2 — 作者/运维 UX 打磨

这阶段解决的是"装得上了，但用得不顺"：

功能 作用 价值	
细粒度 --extras=op 分包选装，而非全有全无 中 — 移除心理阻力 s,frontend	
doctor 增强：项目 检查 CLAUDE.md 是否包含规定章节 (what/how-to-run 中 — 强制执行"短而硬的 文件语义校验 /priorities/guardrails) 骨架"哲学	
交互式 init (无参数时 复用已在 package.json 的 prompts 依赖 中 — 降低首装门槛 prompt)	
--json 输出模式 所有命令支持机器可读输出 中 — 为 CI 集成铺路	
company-cc ci 在 CI 里跑，项目文件缺失/过期时 exit 1 中 — 让团队无法偷懒	

预估工作量：~3-4 天。

Phase 3 — 可扩展性（战略下注）

这是决定这个 harness 能不能真正"company-wide"的关键。当前所有内容都在仓库里硬编码；如果各团队要加自己的 rules/skills，只能 fork。

F: 团队 overlay 层

- 新增 .company-cc.json 配置文件 (user 级 + project 级)
- 支持额外的模板源:
 - 本地路径 (./team-templates/)
 - Git URL (git+https://github.com/team/harness-addons)
 - npm 包 (@team/harness-addons)
- 覆盖规则: baseline → team overlay → personal overlay, 后者赢
- manifest 扩展: 记录每个文件的 source, 避免跨源冲突

G: 自定义 target 适配器

- 当前 targets.js 硬编码 claude/codex。新增 Cursor、Aider、Gemini 的呼声只会越来越多
- 设计 target 注册接口 (plugin via package.json field 或 config 文件)
- 配合 F 使用: 第三方可以发布 @company/harness-cursor-adapter, 不用改核心

为什么 Phase 3 放在后面: 这两个改动触及核心数据模型 (manifest 从单源变多源)。必须在 Phase 1/2 完成后验证设计稳固, 再动结构。

预估工作量: F 需要 1-2 周 (含设计讨论), G 需要 3-5 天。

Phase 4 — 锦上添花

- backup / rollback — 每次 update 前快照到 ~/.claude/.company-cc-backups//, 支持回滚上一次 update
- 模板作者 linter — 校验 skill frontmatter、@rule import 完整性、settings.json schema。可以只作为 dev 工具存在 test/ 下, 不必暴露给最终用户
- 显式拒绝的: telemetry、技能注册中心、CLI 自更新 — 收益/复杂度不划算

推荐的下一步决策

建议你现在只拍板两件事:

1. 是否同意 Phase 1 的四个命令作为下一个 sprint? 它们独立、可发、填补明显空白, 且不动核心模型。
2. Phase 3 的 overlay 层是否真的是你想走的方向? 如果团队定制化不是 near-term 需求, 可以一直停在 Phase 2, 保持项目精简; 如果是, Phase 1 的设计就要为它预留空间 (比如 manifest schema 里提前加 source 字段)。

我可以立即用 /spec:propose 把 Phase 1 拆成一个完整的 OpenSpec change 给你评审, 或者先聚焦其中一个命令 (status 最独立, 适合作为切入点)。你选哪个方向?